Parallel computer architecture: It refers to the organization, structure, and interconnection of processing elements, memory, and communication mechanisms used to support parallel execution.

Why architecture had to evolve?

Earlier computers:

- Single processor
- Sequential execution

Problems:

- Limited speed
- Poor scalability
- Energy inefficiency

Architectures evolved to support parallelism efficiently.

Evolution of parallel computing architecture:

Stage 1: Sequential (Von Neumann) Architecture

Characteristics:

- Single CPU
- Single memory
- One instruction at a time

Limitation:

- No parallel execution
- Performance bound by clock speed

This architecture cannot support HPC.

Stage 2: Pipelined Architecture

Idea

- Overlap instruction execution stages
- Instruction-level parallelism

Example:

- Fetch → Decode → Execute → Write

Limitation

- Still single instruction stream

- Limited speedup

Stage 3: Vector Architecture

Idea

- Same instruction operates on multiple data
- Data-level parallelism

Used In

- Scientific computing
- Early supercomputers

Stage 4: Shared Memory Multiprocessors

Idea

- Multiple processors
- Shared global memory

Advantage

- Easier programming
- Fast communication

Limitation

- Scalability issues

Stage 5: Distributed Memory Architectures

Idea

- Each processor has its own memory
- Message passing for communication

Advantage

- Highly scalable
- Fault tolerant

Used in

- Clusters
- Supercomputers

Stage 6: Hybrid Architectures (Modern)

Combination of:

- Multicore CPUs
- GPUs
- Distributed nodes

This is the dominant architecture today.

Evolution Summary:

Stage	Architecture	Parallelism Type
Sequential	Von Neumann	None
Pipelined	ILP	Instruction
Vector	SIMD	Data
Shared memory	SMP	Thread
Distributed	Cluster	Process
Hybrid	CPU + GPU	Multi-level

Flynn's classification: As parallel computers evolved, many different architectures appeared. Michael J. Flynn proposed a simple and systematic way to classify them based on how instructions and data are handled.

The core idea behind Flynn's classification:

Flynn classified computers based on two dimensions:

1. Instruction Stream – How many instructions are executed at the same time?
2. Data Stream – How many data items are processed at the same time?

Each stream can be Single (S) or Multiple (M).

The four classes of Flynn's taxonomy:

Instruction Stream	Data Stream	Class
Single	Single	SISD
Single	Multiple	SIMD
Multiple	Single	MISD
Multiple	Multiple	MIMD

SISD – Single Instruction, Single Data:

A SISD system executes one instruction at a time on a single data item.

Characteristics:

- Sequential execution
- Single processor
- No parallelism

Examples:

- Traditional uniprocessor computers
- Early computers

Limitation:

- Very poor performance for large problems

SISD is not suitable for HPC.

SIMD – Single Instruction, Multiple Data:

A SIMD system executes the same instruction simultaneously on multiple data items.

Key Idea

- One control unit
- Many processing elements
- Data-level parallelism

Examples

- Vector processors
- GPUs
- SIMD instructions in CPUs

Best For

- Matrix operations
- Image processing
- Scientific computing

MISD – Multiple Instruction, Single Data:

A MISD system executes multiple instructions on the same data stream.

In reality

- Very rare
- Mostly theoretical

Possible Use

- Fault-tolerant systems
- Redundant computation

Almost no commercial systems use pure MISD.

MIMD – Multiple Instruction, Multiple Data:

A MIMD system executes multiple instructions simultaneously on multiple data streams.

Key features

- Fully flexible
- Independent processors
- Most powerful model

Examples

- Multicore CPUs
- Clusters
- Supercomputers

Most modern parallel systems are MIMD.

Comparison:

Feature	**SISD**	**SIMD**	**MISD**	**MIMD**
Instruction streams	1	1	Many	Many
Data streams	1	Many	1	Many
Parallelism	None	Data	Rare	Task + Data
Examples	Old PCs	GPU	Rare	Clusters

Coupling: It refers to the degree of closeness and interaction between processing elements (processors or cores) in a parallel computer system, which is how tightly or loosely processors are connected, how fast they communicate and how much memory they share. Coupling determines communication speed, memory sharing, programming difficulty and scalability.

Types of coupling: Parallel systems are broadly classified into

1. Tightly Coupled Systems
2. Loosely Coupled Systems

A **tightly coupled system** is one where multiple processors share a common memory and communicate via that shared memory.

Characteristics

- Shared global memory
- Fast communication
- Low latency
- Single operating system

Examples

- Multicore processors

- Symmetric Multiprocessors (SMP)

Advantages

- Easy to program
- Fast data sharing
- Low communication overhead

Limitations

- Limited scalability
- Memory contention
- Cache coherence overhead

A **loosely coupled system** consists of independent computers, each with its own memory, connected through a network.

Characteristics

- Distributed memory
- Message passing
- Higher latency
- Multiple operating systems

Examples

- Clusters
- Supercomputers
- Cloud systems

Advantages

- Highly scalable
- Fault tolerant
- Cost effective

Limitations

- Programming is harder
- Communication overhead
- Network latency

Comparison: Tightly vs Loosely Coupled

Aspect	**Tightly Coupled**	**Loosely Coupled**
Memory	Shared	Distributed
Communication	Shared memory	Message passing
Latency	Low	High

Aspect	Tightly Coupled	Loosely Coupled
Scalability	Limited	High
Programming	Easier	Harder
Examples	Multicore CPUs	Clusters

Semi-Coupled (Hybrid) Systems:

Idea

- Combination of tight and loose coupling
- Common in modern HPC systems

Example:

- Multicore CPUs (tight)
- Connected via network (loose)

Most modern supercomputers are hybrid.

Why memory access matters in parallel computers?

Processors need data to compute. How they access memory determines speed, scalability and programming complexity.

Hence, parallel computers are classified based on memory access models.

Major memory access models:

Parallel architectures are broadly classified into:

1. UMA – Uniform Memory Access
2. NUMA – Non-Uniform Memory Access
3. Distributed Memory

UMA (Uniform Memory Access): In a UMA system, all processors access shared memory with equal access time.

Characteristics

- Shared global memory
- Equal latency for all processors
- Usually tightly coupled

Examples

- Symmetric Multiprocessors (SMP)
- Small multicore systems

Advantages

- Simple programming
- Predictable performance
- Easy data sharing

Limitations

- Poor scalability
- Memory bottleneck
- Cache coherence overhead

NUMA (Non-Uniform Memory Access): In a NUMA system, processors have local memory, and access time depends on memory location.

Characteristics

- Physically distributed memory
- Logically shared address space
- Local access is faster than remote

Examples

- High-end servers
- Modern multiprocessor systems

Advantages

- Better scalability than UMA
- Reduced memory contention
- High performance

Limitations

- More complex programming
- Performance depends on data placement

Distributed Memory Systems: In a distributed memory system, each processor has private memory, and processors communicate using message passing.

Characteristics

- No shared memory
- Explicit communication
- Loosely coupled

Examples

- Clusters
- Supercomputers

Advantages

- Very high scalability
- Fault tolerant
- Cost effective

Limitations

- Complex programming
- Communication overhead

Comparison:

Feature	**UMA**	**NUMA**	**Distributed**
Memory	Shared	Shared (non-uniform)	Private
Access time	Uniform	Non-uniform	Local only
Scalability	Low	Medium	High
Programming	Easy	Moderate	Hard
Examples	SMP	High-end servers	Clusters

Vector Processor: It is a processor that can execute a single instruction on an entire vector (array) of data elements simultaneously.

Key idea:

- One instruction
- Many data elements
- Data parallelism

Why vector processors were needed?

Scientific problems often involve:

- Large arrays
- Repetitive calculations

Key components of a vector processor:

Vector Registers: Store multiple data elements

Vector Functional Units: Perform operations on vectors

Vector Control Unit: Issues vector instructions

Types of vector processing:

Memory-to-Memory

- Operates directly on memory
- Used in early systems

Register-to-Register

- Operates on vector registers
- Faster and more efficient

Vector processing techniques:

Pipelining: Overlap execution stages
Chaining: Output of one vector operation feeds another
Masking: Conditional execution on vector elements

Advantages of vector processors:

- Extremely fast for numerical computation
- Reduced instruction overhead
- Efficient use of memory bandwidth

Limitations of vector processors:

- Poor performance for irregular data
- Expensive hardware
- Limited flexibility

Comparison:

Aspect	**Scalar Processor**	**Vector Processor**
Operation	One element	Many elements
Instructions	Many	Few
Performance	Low	Very high
Parallelism	None	Data-level

Superscalar processor: It is a processor that can execute more than one instruction per clock cycle by dispatching multiple instructions to different functional units simultaneously.

Key idea:

- Multiple instructions
- Same clock cycle
- Instruction-Level Parallelism (ILP)

Why superscalar processors were needed?

Even in sequential programs:

- Many instructions are independent
- Waiting wastes CPU time

Superscalar processors:

- Exploit this independence
- Improve performance without increasing clock speed

Scalar vs Superscalar Processing:

Aspect	Scalar Processor	Superscalar Processor
Instructions per cycle	1	>1
Parallelism	None	Instruction-level
Hardware complexity	Low	High
Performance	Limited	High

How superscalar processors work?

Key Components

- Multiple execution units (ALU, FPU)
- Instruction fetch & decode units
- Dynamic scheduling hardware

Execution Steps

1. Fetch multiple instructions
2. Decode them in parallel
3. Dispatch to different functional units
4. Execute simultaneously
5. Commit results in order

Key Techniques Used

Out-of-Order Execution

- Instructions execute as soon as operands are ready

Register Renaming

- Avoids false dependencies

Branch Prediction

- Reduces pipeline stalls

Advantages of superscalar processors:

- Improved single-thread performance
- Efficient use of hardware
- No change required in program

Limitations of Superscalar Processors:

- Complex hardware
- Limited by instruction dependencies
- Diminishing returns (ILP wall)

Relation to Parallel Computing:

- Superscalar means implicit parallelism
- Programmer does not write parallel code
- Compiler and hardware handle it

Comparison with vector processors:

Feature	**Vector Processor**	**Superscalar Processor**
Parallelism	Data-level	Instruction-level
Instructions	Vector	Scalar
Control	Single instruction	Multiple instructions
Programming	Explicit	Implicit

Very Long Instruction Word (VLIW) processor: It is a processor architecture in which multiple independent operations are packed into a single long instruction, and executed simultaneously.

Key idea:

- One very long instruction
- Contains multiple operations
- Executed in parallel

Why VLIW processors were introduced?

Superscalar processors:

- Detect parallelism at runtime
- Require complex hardware

VLIW processors:

- Shift complexity to compiler

- Simplify hardware

How VLIW processor works?

Each field in VLIW instruction:

- Sent to a different functional unit
- Executed in the same cycle

Advantages of VLIW processors:

- Simpler hardware
- Lower power consumption
- Deterministic execution
- High performance when compiler is good

Limitations of VLIW Processors:

- Requires very advanced compiler
- Poor backward compatibility
- Wasted slots if no parallelism
- Not suitable for irregular code

Comparison:

Feature	**Scalar**	**Superscalar**	**VLIW**
Instructions per cycle	1	Many	Many
Parallelism control	None	Hardware	Compiler
Hardware complexity	Low	High	Low
Compiler role	Basic	Moderate	Critical

Interconnection network: It is a communication system that connects processors, memory modules, and computers to enable data exchange in parallel systems.

In simple words: It is the communication backbone of a parallel computer.

Why interconnection networks are needed?

- Multiple processors need to:
    - Access memory
    - Exchange data
- Direct wiring is:
    - Expensive
    - Unscalable

Networks provide structured, scalable communication.

Types of interconnection networks:

- Processor–Memory Interconnection
- Computer–Computer Interconnection
- Direct Interconnection of Computers

Processor–Memory Interconnection:

Purpose: Connects processors to shared memory modules**.**

Common Techniques

Bus-Based Interconnection

- Single shared bus
- Simple but not scalable

Used in small SMP systems.

Crossbar Switch

- Direct connection between every processor and memory
- Very fast

Expensive for large systems.

Multistage Interconnection Networks (MIN)

- Multiple switching stages
- Balance between cost and performance

Common in large shared-memory systems.

Computer–Computer Interconnection:

Purpose: Connects multiple independent computers (nodes).

Common Network Types

Network	Description
Ethernet	Low cost, higher latency
InfiniBand	High speed, low latency
Myrinet	HPC-oriented

Used in:

- Clusters
- Distributed systems

Direct Interconnection of Computers:

Idea: Each computer is directly connected to others in a structured topology.

Common Topologies:

Mesh:

- Nodes arranged in grid
- Simple, scalable

Torus:

- Mesh with wrap-around connections
- Reduced latency

Hypercube:

- High connectivity
- Complex wiring

Tree:

- Hierarchical
- Widely used in modern clusters

Comparison of Interconnection Methods:

Aspect	Bus	Crossbar	MIN	Direct Networks
Scalability	Low	Low	Medium	High
Cost	Low	High	Medium	Medium
Latency	High	Low	Medium	Low